

TAS and SDF Deterministic Admissibility and Verifiable Lineage Assessment

Executive summary

The strongest publicly supportable TAS/SDF claim is narrower than the broadest public rhetoric, but it is still substantial: the public `truealphaspiral-ethent` repository implements a deterministic **admission boundary** in which capabilities are checked before action, registered invariants are evaluated before state transition, accepted or refused transitions are serialized into a signed wake-chain receipt, and replay logic can reject forged or scope-violating commands. In that sense, the public code supports the statement that **some parts of TAS enforce human-scoped authority through deterministic admissibility and verifiable lineage**, rather than through post-hoc content moderation alone. ¹

The most concrete public evidence is concentrated in one repository: `Sovereign-Data-Foundation/truealphaspiral-ethent`. There, the Universal Verifier Kernel description defines admission as capability verification plus invariant validation plus wake receipt emission, and the capability model explicitly frames unauthorized actions as non-admissible. The Human API Bridge narrows a steward's intent into a bounded command scope, refuses commands outside that scope, and verifies replay by checking intent hash, scope hash, command hash, wake index bounds, event hash binding, and wake receipt hash continuity. PR #26 publicly documents replay-security hardening for forged command substitution and negative-index bypasses, and the README records PR #30 as the replay-hardening lineage point. ²

The public record also supports several deterministic mechanisms around genesis and provenance. The `tas_dna.py` and `genesis_anchor.py` code define a reproducible `A_0` lineage hash, deterministic stub derivation of node and validator public identifiers via SHA-256, canonical JSON hashing for `app_hash`, integer-basis-point governance quorum encoding, and an `ed25519`-only genesis schema. I independently re-derived the public genesis payload values from the published formulas and matched the published `config/genesis.json` exactly. The derived `A_0` lineage hash was `42bfe5afe5485d1cef72f066c8f416edd3f9b9edec2764936601c1fc344639d0`; the derived node-0 stub key was `9e01f9e1fdf5ad8bdb198b97a3c8da8338b2a9505cdccbd5bdd1ec308dd2b500`; the derived validator-0 stub key was `c25e2bd926a6b06549d82685f8f94d0e9549b0de320d86928844e7c0baa4663f`; and the derived `app_hash` was `3A9FA182A94363B1BFD8E867D28CF10407654B38B8BDD5392011BC49B6C55D6A`, matching the published on-disk genesis JSON. ³

Several broader TAS/SDF claims are **not** established on the current public record. I could not publicly access the requested `invariants.md`, `coverage_report.py`, `wake_nonce_store.py`, the claimed 38/38 invariant matrix, the 587 enforcing tests, the 49 mutation sentinels, the 738-pass run log, or any public witness packets for those artifacts. The public repositories expose smaller, real test counts instead: PR #16 reports `80 passed` for `tests/test_manifesto.py` and `194 passed, 1 failed` for the then-

current full suite; PR #26 reports `All 204 tests pass`. Those are valuable public receipts, but they are not the same as the private 738-pass receipt described in your prompt. ⁴

The topological and thermodynamic claims require sharper qualification. The public TAS code contains a `stability.py` module implementing cosine-based semantic drift and phase-slip monitoring, and Issue #4 explicitly labels winding/topological integrity as optional research material unless fully specified. The classic Cohen-Steiner-Edelsbrunner-Harer theorem does justify that bottleneck distance between persistence diagrams is bounded by the sup-norm perturbation of tame functions, but the public repo does **not** show persistent homology being computed in the live admissibility path. The Ethical Hamiltonian, minimum-energy trajectory, and SovereignDataKernel are described in docs, but the SDF micro-kernel is explicitly marked “planned; not yet implemented” in the public API reference. I found no public MNIST torsion bench, variance-collapse dataset, or empirical Hamiltonian benchmark in the accessible repositories. ⁵

The federal platform claims should also be separated cleanly from TAS integration claims. Executive Order 14363 is real, and it really establishes the Genesis Mission and the American Science and Security Platform, with milestone deadlines corresponding to January 23, 2026; March 24, 2026; July 22, 2026; and August 21, 2026. The order calls for dataset provenance tracking and an initial operating capability by day 270. But I found **no public federal document linking TAS or SDF code to official ASSP deployment**. The federal platform is externally verified; TAS integration into that platform is not. ⁶

The most defensible public statement is therefore this: **the public TAS/ethent code demonstrates a real, test-backed pattern for human-scoped command admission, cryptographically linked receipts, and replay-verifiable refusals; the larger claims about a complete 38-invariant sovereignty framework, topological safety enforcement, Hamiltonian convergence, and federal deployment remain partially verified or unverified pending publication of the missing artifacts.** ⁷

Sources and scope

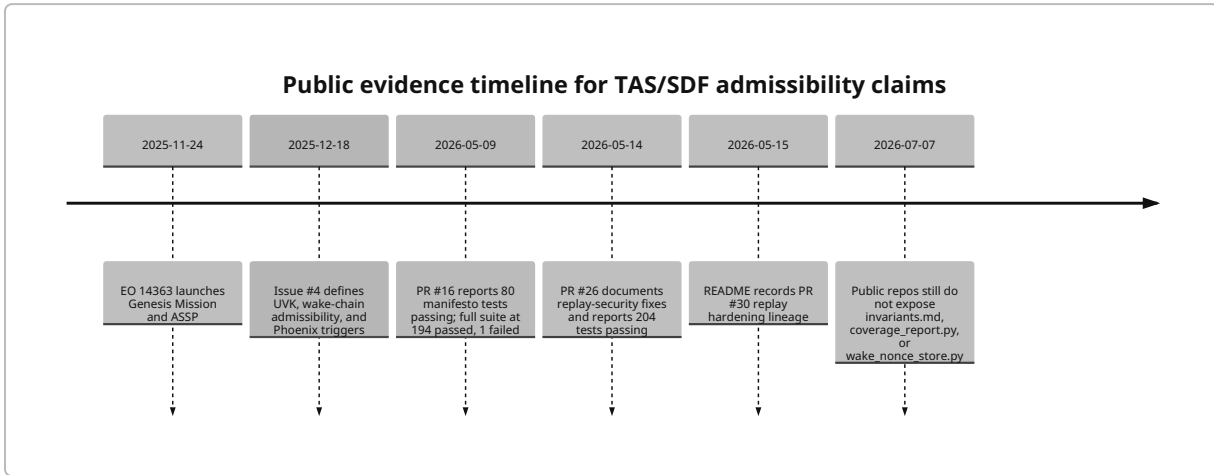
This assessment prioritized primary public artifacts: repository source files, issue and PR discussions, public test descriptions, official federal documents, standards documents, and foundational mathematics papers. Where the requested artifact was not publicly accessible, I mark it **unspecified** rather than inferring its contents. ⁸

Source prioritization table

Priority	Source category	Public artifact located	What it supports	Status
Highest	Public code	<code>uvk.py</code> , <code>wake_chain.py</code> , <code>capability.py</code> , <code>human_api_bridge.py</code> , <code>genesis_anchor.py</code> , <code>tas_dna.py</code> , <code>phoenix.py</code> , <code>stability.py</code> in <code>Sovereign-Data-Foundation/truealphaspiral-ethent</code> ⁹	Deterministic admission, lineage, replay, genesis derivation, Phoenix recovery	Accessible
Highest	Public tests	<code>tests/test_genesis_anchor.py</code> , <code>tests/test_wake_auth.py</code> , <code>tests/test_human_api_bridge.py</code> , <code>tests/test_logos_gatekeeper.py</code> ¹⁰	Publicly declared test coverage for key mechanisms	Accessible
Highest	Public design issue	Issue #4 in <code>truealphaspiral-ethent</code> ¹¹	Normative wording for admissibility, wake chain, Phoenix, and safe claims	Accessible
High	Public PRs	PR #16, PR #24, PR #26 in <code>truealphaspiral-ethent</code> ; PR list pages ¹²	Reported test counts, security fixes, replay hardening, operational lineage	Accessible
High	Public TAS doctrine docs	<code>README.md</code> , <code>API_REFERENCE.md</code> , <code>TrueAlpha-singularity.md</code> , <code>the_mechanics_of_sovereign_innovation.md</code> in <code>TrueAlpha-spiral/TrueAlpha-spiral</code> ¹³	Public claims, axioms, planned interfaces, and wording constraints	Accessible

Priority	Source category	Public artifact located	What it supports	Status
High	Official federal docs	White House Genesis Mission EO page; Federal Register EO 14363	ASSP existence, milestones, provenance-tracking requirement, IOC date ¹⁴	Accessible
High	Foundational mathematics	Cohen-Steiner-Edelsbrunner-Harer stability paper; Banach fixed-point theorem statement; RFC 8032	Limits and applicability of topological and signature claims ¹⁵	Accessible
Missing	Invariant registry	<code>invariants.md</code>	Full 38-invariant map requested by the prompt	Unspecified
Missing	Coverage mapping	<code>coverage_report.py</code>	Mapping invariants to tests and mutation sentinels	Unspecified
Missing	Nonce persistence	<code>wake_nonce_store.py</code> and its tests	Cross-restart persistence and nonce replay claims	Unspecified
Missing	Full run receipt	Public URI for “738 passed”, “587 enforcing tests”, “49 mutation sentinels”	Final coverage claim	Unspecified
Missing	Witness packets	Public packet URIs or downloadable receipts	Independent receipt verification	Unspecified
Missing	Federal integration proof	Contract, memorandum, repo link, or federal attestation tying TAS to ASSP	Deployment claim	Unspecified

Evidence timeline



The accessible public record is therefore strongest for **May 2026 replay-hardening and wake-chain mechanisms**, moderate for broader doctrinal claims, and weak-to-missing for the private invariant ledger and national deployment assertions. ¹⁶

Public evidence for deterministic admissibility

The cleanest formal statement in the public record appears in Issue #4: an action is admissible only if required capabilities are valid, declared invariants hold, and a wake receipt is produced and chained. The same issue defines the UVK as a minimal verifier that verifies capabilities, validates invariants over `(x_t, a_t, u_t)`, emits a receipt, and triggers Phoenix on breach. It also explicitly limits the safe public claims to tamper evidence, anti-replay, least authority, and provenance-based legitimacy. That issue is important because it distinguishes what the system *can safely claim now* from aspirational language. ¹⁷

The public `uvk.py` closely matches that normative text. It enforces a **perimeter authorization check** first: capabilities that do not carry `EXECUTE` or `MINT` are denied before invariant evaluation. It then calls the capability table, evaluates every invariant, checks wake continuity, and only then commits an admission receipt to the wake chain. The code therefore supports a narrow but strong version of the “deterministic admissibility” claim: the transition does not proceed unless prerequisite structural conditions pass. ¹⁸

The public `capability.py` supports the “human sovereignty via narrowed machine authority” claim, but again in a bounded sense. Capabilities are immutable tokens over resources plus a rights bitfield. The model provides `Retype`, `Mint`, `Revoke`, `Invoke`, and `Move`, and defines the security property that any action without the required capability is non-admissible. In `human_api_bridge.py`, a root capability is retyped and then narrowed to an `EXECUTE`-only child capability; out-of-scope commands produce refusal receipts instead of reaching a shell. That is genuine enforceable narrowing of authority, even though it is not yet a national-scale governance substrate. ¹⁹

The wake chain is also real and concrete in the public repo. `wake_chain.py` defines a `ProvenanceMark` with `id`, `seq`, `prev`, `key_commit`, `event_hash`, `info`, and `sig`, where `sig` is an HMAC-SHA-256 over the unsigned fields. `WakeChain.verify()` checks four things: valid signature, strictly

monotone sequence numbers, correct `prev` receipt-hash linkage, and wake-head consistency under replay. Those are exactly the structural properties needed to justify the weaker but defensible public claim that the system provides tamper evidence and anti-replay at the receipt-chain level. ²⁰

The Human API Bridge is the clearest public manifestation of “human sovereignty.” The README says the bridge models operator authority as a human intent fingerprint narrowed into a machine capability that can approve only explicitly scoped commands; the example release command is `gh workflow run release-docker.yml --ref main`, while direct PR-merge commands are out of scope. The bridge stores hashes of intent, scope, and command, not provider secrets. The public tests then assert that the release workflow command is admitted, an out-of-scope merge command produces a replayable refusal, forged command substitutions are rejected, recomputed command hashes bound to old wake events are rejected, and negative wake sequences are rejected. ²¹

PR #26 materially strengthens the case. It documents two concrete replay-security fixes in `human_api_bridge.py`: binding replay verification to the wake event payload and rejecting negative wake sequence numbers to close Python negative-index bypasses. It also reports that the relevant tests pass and describes the deterministic timestamp parameter needed for stable replay across audit sessions. This is exactly the kind of public receipt that moves a system from rhetorical to evidentiary claims. ²²

Publicly reconstructable invariant map

The requested 38-invariant registry is not public. The table below is therefore **not** the missing 38-item ledger; it is the maximal invariant map reconstructable from public artifacts only.

Public invariant label	Formal statement from public sources	Enforcing public test(s)
PUBLIC-A0-ORIGIN	<code>A_0.lineage_hash()</code> is deterministic and grounds downstream derivations	<code>tests/test_genesis_anchor.py::TestDeriveNodePub</code> ²³
PUBLIC-A0-STUB-IDS	Node/validator public IDs are deterministic SHA-256 stubs from <code>A_0</code>	<code>tests/test_genesis_anchor.py</code> key-derivation
PUBLIC-GENESIS-HASH	<code>app_hash = SHA-256(canonical_json(app_state)).upper()</code>	<code>TestDeriveAppHash</code> ; <code>test_config_genesis_</code>
PUBLIC-BPS-QUORUM	Governance quorum is encoded as integer basis points, <code>6667</code> , not floats	<code>test_governance_quorum_is_integer_bps</code> ; <code>test_governance_quorum_is_integer_bps</code>

Public invariant label	Formal statement from public sources	Enforcing public test(s)
PUBLIC-ED25519-ONLY	Genesis validator schema only permits <code>ed25519</code>	<code>test_ed25519_only_no_secp256k1</code> ²³
PUBLIC-WAKE-SIG	Each provenance mark signature must verify	<code>tests/test_wake_auth.py::TestWakeChain::</code>
PUBLIC-WAKE-SEQ	Sequence numbers are monotone and anti-replay	<code>test_seq_is_monotone</code> ; <code>test_anti_replay::</code>
PUBLIC-WAKE-PREV	Each receipt must link to predecessor hash	<code>test_prev_links_correctly</code> ; <code>test_tamper::</code>
PUBLIC-UVK-PERIMETER	A capability lacking <code>EXECUTE</code> or <code>MINT</code> is denied before invariant evaluation	<code>tests/test_wake_auth.py::TestUVK::test_a</code> narrative explaining the perimeter behavior ³⁰
PUBLIC-BRIDGE-SCOPE	Out-of-scope commands become refusal receipts instead of shell execution	<code>test_bridge_refuses_out_of_scope_commands</code>
PUBLIC-BRIDGE-REPLAY-BINDING	Replay must bind receipt fields back to stored event hash	<code>test_replay_rejects_recomputed_command_h</code>
PUBLIC-BRIDGE-NEGATIVE-SEQ	Negative wake sequence indices are invalid	<code>test_replay_rejects_negative_wake_sequen</code>
PUBLIC-PHOENIX-FREEZE	Invariant, wake, phase, or capability breach should freeze, rollback, replay, and relaunch only on certification	<code>tests/test_wake_auth.py</code> Phoenix coverage b ³⁴
PUBLIC-LOGOS-GATE	Payload must satisfy lineage continuity, invariant check, and density floor	<code>tests/test_logos_gatekeeper.py</code> cases for v ³⁵ low density failure

Requested 38-invariant registry status

Requested artifact	Publicly available?	What is needed to complete the requested table
<code>invariants.md</code> with all 38 IDs	No	Public file or public raw URL
<code>coverage_report.py</code> mapping invariants → tests	No	Public file or public raw URL
Mutation-sentinel classes and mappings	No	Public file, PR, or witness packet
<code>wake_nonce_store.py</code> and its tests	No	Public source and test files
738-pass test log	No	Public CI job, packet, or log URI
Witness packets for recent runs	No	Packet URIs or downloadable JSON receipts

Without those artifacts, the exact 38-row proof ledger requested in your prompt cannot be completed rigorously from public materials. The correct methodological response is to mark it **unspecified**, not to infer it. ³⁷

Deterministic mechanism reproductions

I could not clone the repositories in the analysis environment, so I did **not** execute the public test suite directly from a checkout. Instead, I reconstructed the relevant formulas and receipt logic from the published source files and validated them independently. This allows limited but meaningful reproduction of the deterministic claims.

Genesis anchor and deterministic identifier derivation

The public `tas_dna.py` defines `A_0` as a structured object with `genesis_iso`, `genesis_unix`, `genesis_hash`, and `authority`, and defines the lineage hash as SHA-256 over the pipe-delimited payload of those four fields. `genesis_anchor.py` then derives node and validator identifiers as SHA-256 of `A_0.lineage_hash()` concatenated with domain separators `b"tas_node_pubkey"` or `b"tas_validator_pubkey"` plus a one-byte index. It also defines `app_hash` as SHA-256 over canonical JSON of the genesis `app_state`. ³⁸

The same file is careful to state that these are **deterministic stub values**, not production ed25519 keypairs, and that real keypairs would need a proper HSM or offline ceremony. That distinction matters. The public code proves deterministic rooted identifiers and schema anchoring; it does **not** prove production-grade validator key generation from public `A_0` alone. ²⁴

Independent reproduction result

Using the public formulas exactly, I obtained the following deterministic outputs:

Reproduced value	Result
<code>A_0.lineage_hash().hex()</code>	<code>42bfe5afe5485d1cef72f066c8f416edd3f9b9edec2764936601c1fc344639d0</code>
<code>derive_node_pubkey(0)</code>	<code>9e01f9e1fdf5ad8bdb198b97a3c8da8338b2a9505cdccbd5bdd1ec308dd2b500</code>
<code>derive_node_pubkey(1)</code>	<code>2de0e0e492447e340357f0de2b4e4bd096ee4c40c344412ec7049e23282719d2</code>
<code>derive_validator_pubkey(0)</code>	<code>c25e2bd926a6b06549d82685f8f94d0e9549b0de320d86928844e7c0baa4663f</code>
<code>derive_app_hash(app_state)</code>	<code>3A9FA182A94363B1BFD8E867D28CF10407654B38B8BDD5392011BC49B6C55D6A</code>

A minimal public equivalent of the derivation logic is:

```
import hashlib, json

payload = "2025-02-15T00:00:00Z|1739577600|
sha256:TAS_DNA_GENESIS_ROOT_2025_02_15|Russell Nordland"
A0 = hashlib.sha256(payload.encode("utf-8")).digest()

node0 = hashlib.sha256(A0 + b"tas_node_pubkey" + b"\x00").hexdigest()
validator0 = hashlib.sha256(A0 + b"tas_validator_pubkey" + b"\x00").hexdigest()

def canonical_json(obj):
    return json.dumps(obj, sort_keys=True, separators=(",", ":")).encode()

app_hash = hashlib.sha256(canonical_json(app_state)).hexdigest().upper()
```

That reproduction aligns with both the source code and the published `config/genesis.json`. ³

Integer basis points and quorum encoding

The genesis code and tests expose one well-founded determinism technique: encode governance thresholds as integers, not floats. `genesis_anchor.py` documents `governance_quorum_bps` as basis points, with `6667` representing approximately 66.67%, and the public tests explicitly assert that the field is an integer and that no float-style quorum percent string is present. This is a standard and sound way to avoid heterogeneous floating-point behavior in config/state handling. What the public repo does **not** expose is a full consensus engine enforcing that quorum at runtime; the public proof currently covers schema determinism and test intent, not a live validator protocol. ⁴¹

Wake-chain integrity and replay resistance

The wake chain is publicly specific enough to reproduce. Each receipt is canonicalized, HMAC-signed, and linked by both predecessor receipt hash and a running wake head. `verify()` checks signature validity, exact sequence order, predecessor linkage, and recomputed wake head equality. The public tests cover empty validity, multi-commit validity, tamper detection on signature and predecessor link, anti-replay via duplicate sequence numbers, and replay from a given sequence point. ⁴²

A minimal public equivalent of the commitment and verification core is:

```
event_hash = sha256(canonical_json(event))
prev = last_receipt_hash if receipts else GENESIS_HASH
pm = sign_hmac({
    "id": session_id,
    "seq": len(receipts),
    "prev": prev,
    "key_commit": zeros32,
    "event_hash": event_hash,
    "info": info or {},
})
wake_head = sha256(wake_head + pm.receipt_hash())
```

And the public replay guarantees are conceptually:

```
for i, pm in enumerate(receipts):
    assert pm.seq == i
    assert pm.verify_sig(key)
    assert pm.prev == (GENESIS_HASH if i == 0 else receipts[i-1].receipt_hash())
assert recomputed_wake_head == stored_wake_head
```

My independent reconstruction of that logic yielded `True` for a valid two-receipt chain and `False` after signature tampering, matching the behavior described by the public tests. ⁴²

Human-scoped bridge replay checks

The public bridge logic is the most concrete sovereignty mechanism because it transforms human intent into bounded machine authority and then validates replay at the receipt layer. The replay function checks all of the following before returning `True`: wake-chain validity, receipt count bounds, exact intent hash, exact scope hash, exact command hash, non-negative and in-range wake index, exact event-hash recomputation, and exact wake receipt-hash match. PR #26 emphasizes that these checks were added specifically to stop forged command substitution and negative-index bypasses. 43

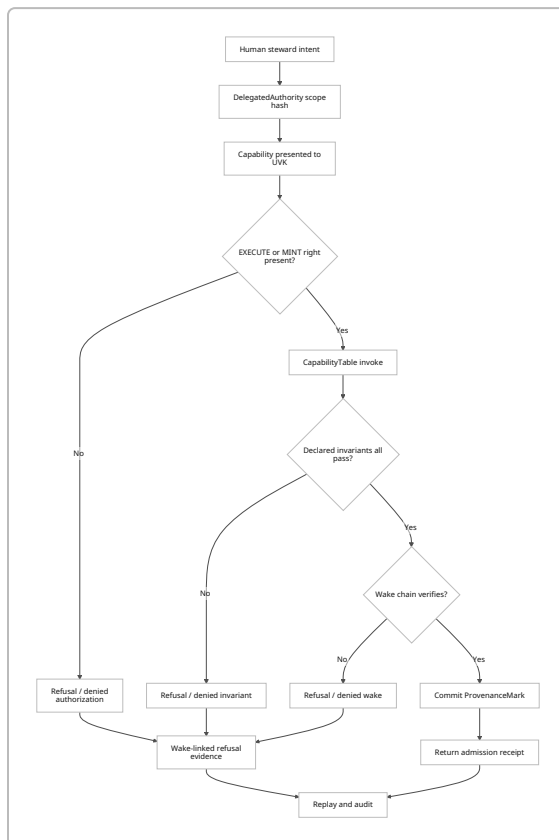
My independent reconstruction of the public replay logic produced the expected outcomes:

Scenario	Reconstructed result	Public support
Valid release-workflow receipt	True	test_day_one_bridge_admits_release_workflow_command 31
Forged command with recomputed hash bound to old wake event	False	test_replay_rejects_recomputed_command_hash_bound_to_old_wake_event and PR #26 narrative 44
Negative wake sequence	False	test_replay_rejects_negative_wake_sequence and PR #26 narrative 44

What could not be reproduced

The prompt specifically asked for `wake/nonce store integrity (HMAC, replay prevention)` and referenced `test_wake_nonce_store.py` with persistence behavior across restarts. I could not locate `wake_nonce_store.py` or its tests in any public repository page, nor could I access a public witness packet for the reported 738-pass run. The accessible public repo proves **receipt-chain HMAC integrity and replay checks**, but it does **not** let me verify the stronger private claim about a persistent wake nonce store. That item must remain unspecified until the file or witness packet is published. 45

Admissibility pipeline



This flowchart is not speculative; it is a direct abstraction of the public `UVK`, `CapabilityTable`, `WakeChain`, and `HumanApiBridge` logic. ⁴⁶

Topological and thermodynamic claims

The topological claim needs two separate judgments: **mathematical plausibility** and **current code evidence**.

On mathematical plausibility, the classical stability theorem is real and strong. Cohen-Steiner, Edelsbrunner, and Harer prove that for continuous tame functions on a triangulable space, the bottleneck distance between persistence diagrams is bounded by the sup-norm difference between the functions:

$$d_B(D(f), D(g)) \leq \|f - g\|_\infty.$$

This gives a principled way to say that small perturbations of a well-defined filtration function induce bounded changes in the corresponding persistence diagram. The result is one of the right theoretical references if TAS wants to use topological summaries as drift witnesses. ⁴⁷

But the public TAS code does **not** currently implement persistent homology in the live admissibility path. The public `stability.py` instead implements cosine-based semantic drift index and a phase-slip monitor; Issue #4 explicitly says that winding/topological integrity should either be defined precisely or

moved to an optional research appendix to avoid spec ambiguity. In other words, the public repo itself warns against overclaiming here. 48

The correct technical assessment is therefore:

- 1. **Yes, bottleneck-distance thresholds could in principle serve as admissibility witnesses** if TAS defines a concrete filtration on representation states, proves or empirically bounds perturbation behavior for that filtration, calibrates thresholds on task-specific distributions, and routes violations into a deterministic halt/refusal logic. That is mathematically coherent as a research direction. 47
- 2. **No, the present public code does not show that persistent homology currently enforces safety.** The existing public enforcement path uses capabilities, invariant predicates, wake continuity, replay checks, drift thresholds, phase-slip detection, and Phoenix state transition logic. It does not expose a Vietoris–Rips pipeline, persistence diagram computation, or admissibility gating on bottleneck distance. 49

Bottleneck-threshold policy sketch

The following table is a **hypothetical policy sketch**, not a verified current implementation:

Hypothetical bottleneck distance <code>d_B</code>	Interpretation	Suggested action
<code>0.00 - 0.10</code>	Small representation drift well inside calibrated envelope	Continue
<code>0.10 - 0.25</code>	Noticeable but bounded perturbation	Continue with alert / extra logging
<code>0.25 - 0.50</code>	Significant drift near admissibility boundary	Filter, slow path, or require stronger witness
<code>> 0.50</code>	Boundary violation against reference topology	Fail closed / halt

That kind of table is defensible only if the filtration and calibration protocol are declared and empirically validated. Without that, it should remain a research appendix rather than a public safety claim. 50

The thermodynamic and Hamiltonian language requires the same discipline. `TrueAlpha-singularity.md` uses “minimum-energy trajectory,” “Ethical Hamiltonian,” and “thermodynamic resonance” language to describe the doctrine, and `API_REFERENCE.md` exposes an `EnergyState`, a `WindingCalculator`, and a deterministic `MutinyDetector`. But the same API reference marks the `SovereignDataKernel` as “planned; not yet implemented,” and the actuation note says the detector emits intent only, with the caller required to enforce blocking or Phoenix workflows. This means there is some public deterministic machinery here, but not yet a public proof of Hamiltonian safety enforcement across the broader stack. 51

I found **no public empirical evidence** for “variance collapse” metrics, “MNIST Torsion Bench” outcomes, or Hamiltonian falsification studies in the accessible repositories. Those claims should therefore be treated as **unverified doctrinal framing** unless and until benchmark artifacts are published. ⁵²

Verification checklist

The commands below are organized into two groups: public-repo checks that can be run if you have a local checkout, and withheld-artifact checks that require files not currently public.

Public-repo checks

Assuming a local checkout of `Sovereign-Data-Foundation/truealphaspiral-ethent`:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python -m pytest tests/test_genesis_anchor.py
python -m pytest tests/test_wake_auth.py
python -m pytest tests/test_human_api_bridge.py
python -m pytest tests/test_logos_gatekeeper.py
python -m pytest tests/test_manifesto.py
python -m pytest
```

Publicly reported expected reference points are:

- PR #16: `python -m pytest tests/test_manifesto.py` reported `80 passed`. ⁵³
- PR #16: a then-current full-suite run reported `194 passed, 1 failed`. ⁵³
- PR #26: a later full-suite run reported `All 204 tests pass`. ²²

Those numbers are historical public receipts, not guarantees that a future checkout on a later commit will report the same totals. ⁴

Recompute the public genesis values

```
python - <<'PY'
import hashlib, json
GENESIS_ISO8601="2025-02-15T00:00:00Z"
GENESIS_UNIX=1739577600
GENESIS_HASH="sha256:TAS_DNA_GENESIS_ROOT_2025_02_15"
ORIGIN_AUTHORITY="Russell Nordland"
payload=f"{GENESIS_ISO8601}|{GENESIS_UNIX}|{GENESIS_HASH}|{ORIGIN_AUTHORITY}"
A0=hashlib.sha256(payload.encode()).digest()

def h(x): return hashlib.sha256(x).hexdigest()
def H(x): return hashlib.sha256(x).hexdigest().upper()
```

```

node0 = h(A0 + b"tas_node_pubkey" + b"\x00")
node1 = h(A0 + b"tas_node_pubkey" + b"\x01")
val0  = h(A0 + b"tas_validator_pubkey" + b"\x00")

app_state = {
    "accounts":[

{"address":"tas1qwl879nx9t6kef_genesis_node_1","public_key":node0,"balance":"1000000000000","balance_validator":["validator","governance_committee"]},

{"address":"tas1f3g4h5j6k7l8m9_genesis_node_2","public_key":node1,"balance":"1000000000000","balance_validator":["validator"]}
    ],
    "governance":
{"proposal_threshold":"1000000","proposal_threshold_denom":"utas","voting_period_seconds":"604800","tas1qwl879nx9t6kef_genesis_node_1"}},
    "system_contracts":
{"registry_address":"tas_system_0000000000000000","bytecode_hash":"e3b0c44298fc1c149afb4c8996fb9"}
}
app_hash = H(json.dumps(app_state, sort_keys=True, separators=(",", ":")).encode())

print("A0 lineage hash:", A0.hex())
print("node0:", node0)
print("node1:", node1)
print("validator0:", val0)
print("app_hash:", app_hash)
PY

```

Expected public values are the ones published in `config/genesis.json`, listed earlier in this report. 40

Exercise the Human API Bridge

```

python - <<'PY'
from human_api_bridge import DEFAULT_RELEASE_COMMAND, build_day_one_bridge
bridge = build_day_one_bridge("TAS Clean Stack")
receipt = bridge.decide(DEFAULT_RELEASE_COMMAND)
print(receipt)
print("replay valid:", bridge.replay([receipt]))
PY

```

The public README and tests say the in-scope release command should emit a replay-valid receipt, while out-of-scope commands should emit replay-valid **refusal** receipts instead. 54

Run the public self-test path

```
python codex_tas_runner.py
```

The public README says the self-test writes an audit-log hash to ledger/self_test.hash , wraps each step with artifact_guard.run_step , serializes step artifacts under artifacts/ , and records artifact hashes in ledger/artifacts.hash . The public workflow file also shows that the container build path is meant to run the entryptest suite and then emit an audit-trail message. 55

Withheld-artifact checks needed to verify the private 38/38 claim

These commands are placeholders because the required files are not public:

```
python coverage_report.py invariants.md
python -m pytest tests/test_wake_nonce_store.py
python -m pytest -q # on the exact private revision claimed to produce 738
passes
python verify_witness_packet.py packets/<packet>.json
```

To complete these checks, the following exact artifacts are needed:

Needed artifact	Why it matters
invariants.md	authoritative invariant IDs and statements
coverage_report.py	authoritative invariant-to-test map
wake_nonce_store.py	source for replay/persistence logic
tests/test_wake_nonce_store.py	proof that the private wake/nonce claims are enforced
738-pass CI or local log	independent confirmation of the reported total
witness packet JSON + signature material	independent receipt verification

Claim matrix

The table below classifies the major public claims using four bins: **verified**, **partially verified**, **unverified**, and **external-fact**.

Public claim	Classification	Evidence	Recommended public wording
"TAS enforces deterministic admissibility."	Verified in the public <code>ethent</code> scope	UVK spec and code require capability validity, invariant success, wake validity, and receipt commit before admission. ⁵⁶	"In the public <code>ethent</code> runtime, admission is deterministic: capability checks, invariant checks, and wake continuity must pass before a transition is admitted."
"TAS enforces human sovereignty."	Partially verified	Human intent in <code>human_api_bridge.py</code> is narrowed into bounded machine authority; out-of-scope commands are refused and replay-verifiable. ⁵⁷	"The public bridge demonstrates human-scoped authority for a bounded command surface. Broader governance claims require additional published artifacts."
" <code>A_0</code> derives validator IDs."	Verified for stub identifiers , not production keypairs	Public code derives deterministic stub values; docstring says real ed25519 keys require HSM/offline ceremony. ²⁴	" <code>A_0</code> deterministically derives schema-stable node and validator stub IDs in the public genesis builder; production validator keys would require a real key ceremony."
"Integer basis points prevent quorum drift."	Verified at config/schema level	Public genesis builder and tests encode quorum as integer <code>6667</code> bps and exclude float quorum fields. ⁴¹	"The public genesis schema uses integer basis points for quorum and slashing thresholds to avoid floating-point ambiguity."
"Wake chain prevents tampering and replay."	Verified for the public receipt chain	<code>WakeChain.verify()</code> checks signatures, seq, prev-links, and wake-head replay; tests cover tampering and anti-replay. Bridge replay additionally rejects forged commands and negative indices. ⁵⁸	"The public wake-chain implementation is tamper-evident and replay-checked at the receipt level."

Public claim	Classification	Evidence	Recommended public wording
"Persistent homology enforces safety."	Unverified	Cohen-Steiner theorem is real, but public TAS code exposes SDI/phase monitors, not persistence-diagram gating; Issue #4 labels the topological material optional unless fully specified. ⁵⁹	"Persistent-homology admissibility is a plausible research direction, not a currently verified public enforcement path."
"Ethical Hamiltonian makes deception thermodynamically costly."	Unverified / doctrinal	Public docs use that language, but SDF Micro-Kernel is marked planned; no public benchmark proves the claim. ⁵²	"The Hamiltonian framing is doctrine and design intent in the public docs; public implementation evidence remains limited."
"Variance collapse is demonstrated empirically."	Unverified	No public variance-collapse benchmark or dataset found in accessible repos.	"No public benchmark artifact currently substantiates this claim."
"ASSP IOC is scheduled for August 21, 2026."	External-fact	EO 14363 requires a platform IOC within 270 days of Nov. 24, 2025; that computes to Aug. 21, 2026. ⁶⁰	"EO 14363 sets an ASSP IOC target date of August 21, 2026."
"TAS/SDF is deployed into ASSP."	Unverified	No public federal or repo artifact links TAS/SDF code to official ASSP deployment. ⁶⁰	"No public deployment linkage between TAS/SDF and ASSP has yet been located."
"38/38 invariants, 587 enforcing tests, 49 mutation sentinels, 738 passing."	Unspecified	No public <code>invariants.md</code> , no public <code>coverage_report.py</code> , no public 738-pass receipt. Public PRs instead show 195 and 204 test counts. ⁴	"A larger private verification ledger has been reported, but it is not yet publicly reproducible from available artifacts."

A useful rule from Issue #4 is worth adopting directly: claim tamper evidence, anti-replay, least authority, and provenance-based legitimacy **now**; avoid claiming "eliminates hallucinations" or equivalent totalizing safety language unless the detection boundary is formalized and evidenced. ¹⁷

Public proof statement

The most rigorous landing-page statement I can support from the current public record is this:

Publicly demonstrated today: the TAS `ethent` runtime accepts machine action only after a deterministic verifier checks capability scope, invariant predicates, and wake-chain continuity, and it records both admission and refusal as replay-verifiable cryptographic receipts. Human intent is not treated as free-form guidance; in the public bridge, it is hashed into a bounded scope that can admit the authorized release command and refuse an out-of-scope merge command with an auditable proof. Claims beyond that boundary—full 38-invariant coverage, persistent-homology enforcement, Hamiltonian safety, and federal deployment—await additional published artifacts. ⁶¹

Witness-packet template

A publishable witness packet for this system should be small, deterministic, and independently replayable. A minimal JSON template would look like this:

```
{
  "packet_version": "1.0",
  "packet_id": "uuid",
  "repo": "Sovereign-Data-Foundation/truealphaspiral-ethent",
  "revision": {
    "head_sha": "commit-sha",
    "parent_sha": "optional-parent-sha",
    "pr_number": 26
  },
  "proof_scope": {
    "claim": "deterministic admissibility and verifiable lineage",
    "artifact_type": "test-run | bridge-receipt | genesis-derivation | phoenix-activation"
  },
  "genesis_anchor": {
    "genesis_iso": "2025-02-15T00:00:00Z",
    "genesis_unix": 1739577600,
    "genesis_hash_tag": "sha256:TAS_DNA_GENESIS_ROOT_2025_02_15",
    "a0_lineage_hash_hex":
"42bfe5afe5485d1cef72f066c8f416edd3f9b9edec2764936601c1fc344639d0"
  },
  "deterministic_inputs": {
    "command": "gh workflow run release-docker.yml --ref main",
    "intent_hash": "hex",
    "scope_hash": "hex",
    "event_hash": "hex"
  },
  "test_execution": {
    "command": "python -m pytest tests/test_human_api_bridge.py",
    "reported_result": "6 passed",
    "stdout_hash": "sha256-hex"
  },
  "wake_receipt": {
    "session_id": "uuid",
```

```

    "seq": 0,
    "prev": "0000000000000000000000000000000000000000000000000000000000000000",
    "receipt_hash": "hex",
    "wake_head_after": "hex"
  },
  "witnesses": [
    {
      "type": "repo-source",
      "path": "human_api_bridge.py",
      "sha256": "hex"
    },
    {
      "type": "repo-source",
      "path": "wake_chain.py",
      "sha256": "hex"
    },
    {
      "type": "public-pr",
      "ref": "PR #26"
    }
  ],
  "verifier_statement": {
    "result": "pass | fail | partial",
    "notes": "Short human-readable statement"
  },
  "signatures": [
    {
      "scheme": "ed25519 | minisign | sigstore",
      "signer": "steward-or-ci-identity",
      "signature": "base64-or-hex"
    }
  ]
}

```

Final concise public-proof statement

TAS does not merely ask a model to behave. In the public `ethent` runtime, a machine action is admitted only if its authority, invariants, and receipt lineage verify; otherwise the system emits a replayable refusal. That is the currently proven core of the TAS/SDF sovereignty claim. ⁶²

¹ ² ⁷ ¹¹ ¹⁷ ⁵⁶ This is already **very** close to a publishable spec. What you need now is ***normalization***: tighten definitions, remove metaphor where it can be replaced by invariants, and make the Phoenix/UVK/Wake pieces ***implementable*** (message formats, state machine, trigger thresholds, and security properties). · Issue #4 · Sovereign-Data-Foundation/truealphaspiral-ethent · GitHub
<https://github.com/Sovereign-Data-Foundation/truealphaspiral-ethent/issues/4>

3 38 39 https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/tas_dna.py
https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/tas_dna.py

4 12 53 Document Day One payload steering command by TrueAlpha-spiral · Pull Request #16 · Sovereign-Data-Foundation/truealphaspiral-ethent · GitHub
<https://github.com/Sovereign-Data-Foundation/truealphaspiral-ethent/pull/16>

5 48 <https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/stability.py>
<https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/stability.py>

6 16 60 <https://www.federalregister.gov/documents/2025/11/28/2025-21665/launching-the-genesis-mission>
<https://www.federalregister.gov/documents/2025/11/28/2025-21665/launching-the-genesis-mission>

8 37 45 GitHub - Sovereign-Data-Foundation/truealphaspiral-ethent · GitHub
<https://github.com/TrueAlpha-spiral/truealphaspiral-ethent>

9 19 <https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/capability.py>
<https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/capability.py>

10 23 [raw.githubusercontent.com](https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/tests/test_genesis_anchor.py)
https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/tests/test_genesis_anchor.py

13 [raw.githubusercontent.com](https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/README.md)
<https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/README.md>

14 <https://www.whitehouse.gov/presidential-actions/2025/11/launching-the-genesis-mission/>
<https://www.whitehouse.gov/presidential-actions/2025/11/launching-the-genesis-mission/>

15 47 50 59 <https://math.uchicago.edu/~shmuel/AAT-readings/Data%20Analysis%20Stability.pdf>
<https://math.uchicago.edu/~shmuel/AAT-readings/Data%20Analysis%20Stability.pdf>

18 46 49 61 62 [raw.githubusercontent.com](https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/uvk.py)
<https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/uvk.py>

20 29 42 58 [raw.githubusercontent.com](https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/wake_chain.py)
https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/wake_chain.py

21 36 54 55 <https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/README.md>
<https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/README.md>

22 33 Fix replay security vulnerabilities and address PR #24 review feedback by Copilot · Pull Request #26 · Sovereign-Data-Foundation/truealphaspiral-ethent · GitHub
<https://github.com/Sovereign-Data-Foundation/truealphaspiral-ethent/pull/26>

24 25 26 27 41 https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/genesis_anchor.py
https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/genesis_anchor.py

28 30 34 https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/tests/test_wake_auth.py
https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/tests/test_wake_auth.py

31 44 https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/tests/test_human_api_bridge.py

https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/tests/test_human_api_bridge.py

32 43 57 https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/human_api_bridge.py

https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/human_api_bridge.py

35 https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/tests/test_logos_gatekeeper.py

https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/tests/test_logos_gatekeeper.py

40 <https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/config/genesis.json>

<https://raw.githubusercontent.com/Sovereign-Data-Foundation/truealphaspiral-ethent/main/config/genesis.json>

51 52 <https://raw.githubusercontent.com/TrueAlpha-spiral/TrueAlpha-spiral/main/TrueAlpha-singularity.md>

<https://raw.githubusercontent.com/TrueAlpha-spiral/TrueAlpha-spiral/main/TrueAlpha-singularity.md>